

## 类的继承

教 师：冀荣华

单 位：人工智能系



## 主要内容

- 一、继承与派生
- 二、派生类的声明方式
- 三、派生类的构成
- 四、派生类成员的访问属性
- 五、派生类的构造函数和析构函数
- 六、多继承
- 七、基类与派生类的转换



# 一、继承与派生

程序设计的目的

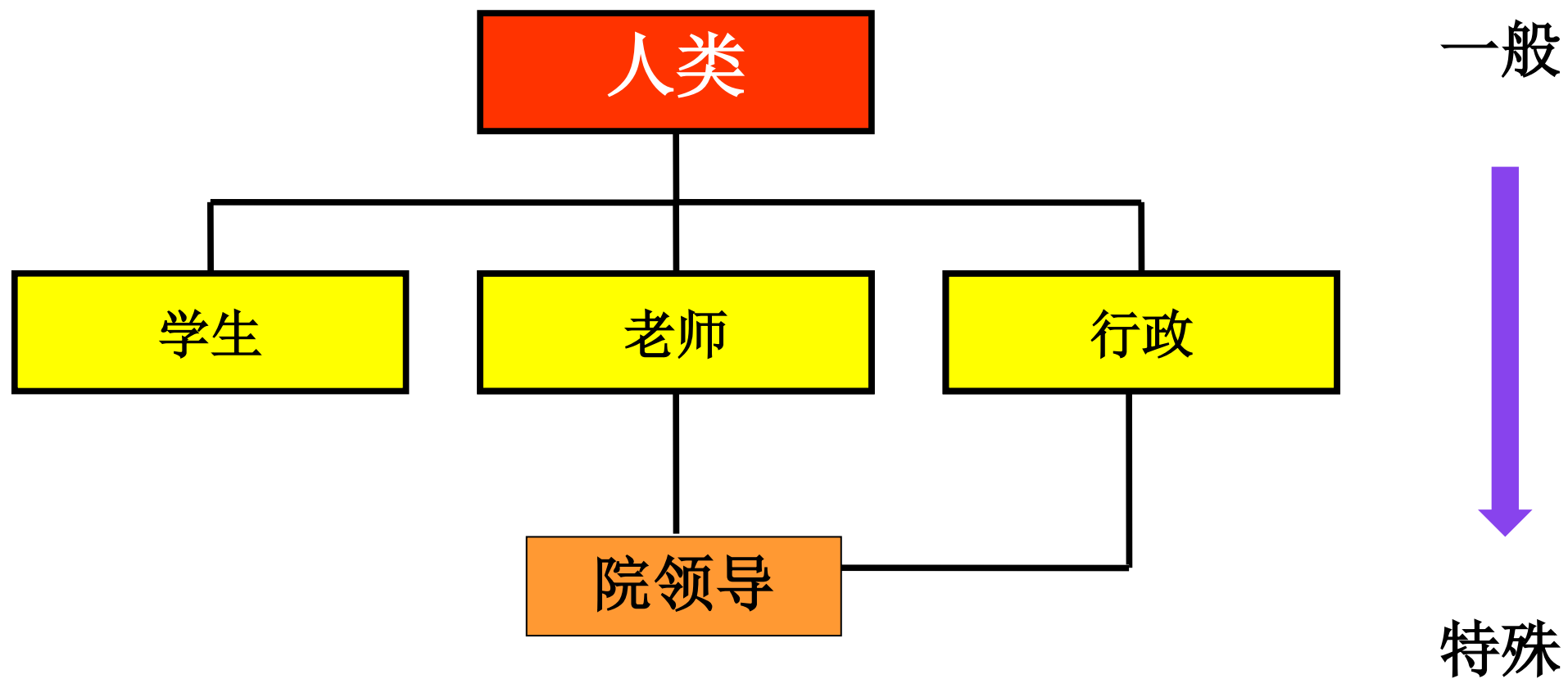
——描述和解决现实世界中的问题

类与对象——反映了个体和群体的关系

继承——描述了一般和特殊的联系



# 一、继承与派生



# 一、继承与派生

```
class Student
{
    public:
        void get_value()
        {cin>>num>>name>>sex;}

        void display()
        {
            cout<<" num: "<<num<<endl;
            cout<<" name: "<<name<<endl;
            cout<<" sex:"<<sex<<endl;
        }
    private:
        int num;
        string name;
        char sex;
};
```



# 一、继承与派生

```
class Gstudent  
{public:
```

```
  
    void get_value_1()  
    {        cin>>age>>addr;        }  
    void display_1()  
    {        cout<<"age:"<<age<<endl;  
            cout<<"address:"<<addr<<endl;    }  
  
private:
```

```
  
    int age;  
    string addr;  
};
```



# 一、继承与派生

```
class Gstudent:public Student
{public:
    void get_value_1()
    {        cin>>age>>addr;        }
    void display_1()
    {        cout<<"age:"<<age<<endl;
        cout<<"address:"<<addr<<endl;
    }
private:
    int age;
    string addr;
};
```



# 一、继承与派生

- 继承机制：

在保持已有类的特性的基础上进行  
更具体、更详细的类的定义

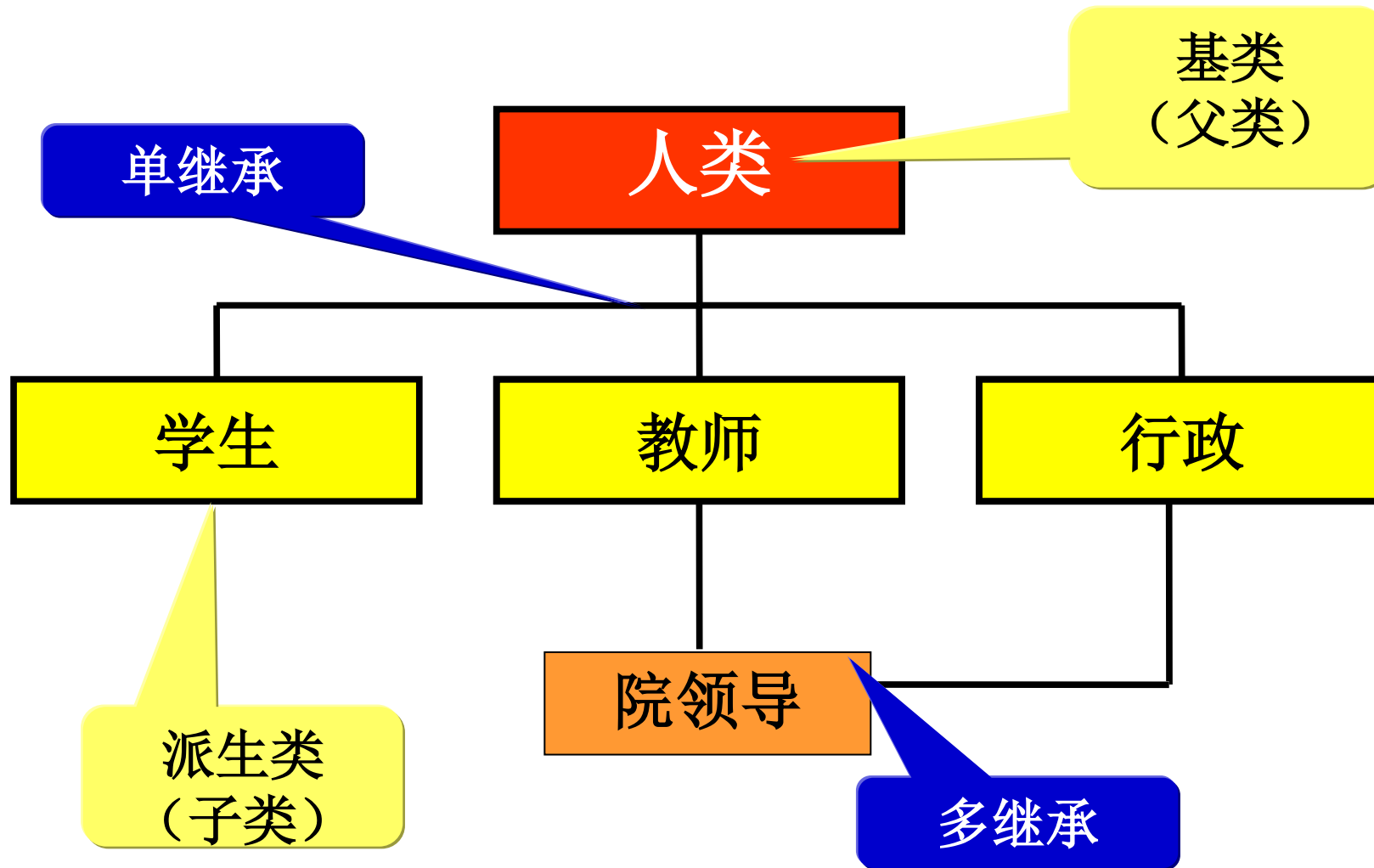
- 继承的意义：

- ★ 代码重用
- ★ 代码可扩充





## 二、派生类的声明方式



## 二、派生类的声明方式

**class** 派生类名: **继承方式** 基类名

{

成员声明;

};

单继承



## 二、派生类的声明方式

```
class Gstudent:public Student
{public:
    void get_value_1()
    {      cin>>age>>addr;      }
    void display_1()
    {      cout<<"age:"<<age<<endl;
          cout<<"address:"<<addr<<endl;
    }
private:
    int age;
    string addr;
};
```



## 三、派生类的构成

### 1、全盘吸收基类成员：

不包括构造函数和析构函数

### 2、改造基类成员：

- 改变基类成员的访问控制
- 对基类成员进行同名隐藏

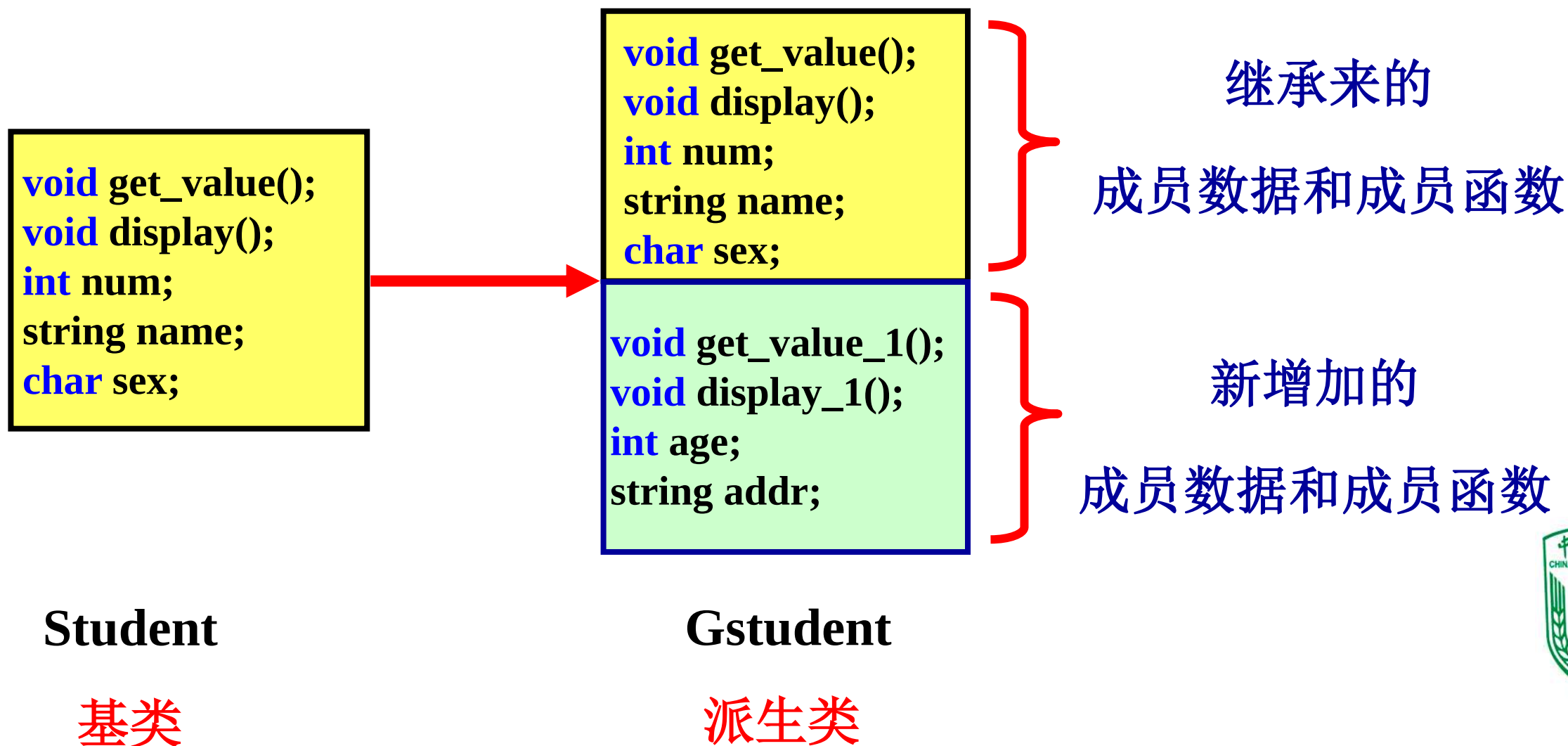
### 3、添加新的成员

- 加入新的构造函数和析构函数（特别的初始化）
- 派生出新的功能



### 三、派生类的构成

类Student派生出类Gstudent



## 三、派生类的构成

来源:

- 一、从基类继承来的数据成员和成员函数。对于继承来的数据成员，即使没有用也不能取消，只能不理睬它们，但允许对一些继承来的成员函数重新定义，即在原有基类的成员函数的基础上，再增加一些操作，以完成派生类特殊要求的操作
- 二、由派生类自己定义的数据成员和成员函数。



# 继承方式

1. 概述
2. 公有继承
3. 私有继承
4. 保护继承



## 四、派生类成员的访问属性

### 1、三种继承方式

- 公有继承
- 私有继承
- 保护继承

### 2、不同继承方式的影响主要体现在：

- 派生类**新增成员**对基类成员的访问权限
- 派生类**对象**对基类成员的访问权限





# 四、派生类成员的访问属性

|             | 不变          | private   | protected   |
|-------------|-------------|-----------|-------------|
| 基类          | 公有派生类       | 私有派生类     | 保护派生类       |
| public成员    | public成员    | private成员 | protected成员 |
| protected成员 | protected成员 | private成员 | protected成员 |
| private成员   | 不可访问        | 不可访问      | 不可访问        |



## 四、派生类成员的访问属性

### 公有继承

| 基类                  | 公有派生类               |
|---------------------|---------------------|
| <b>public</b> 成员    | <b>public</b> 成员    |
| <b>protected</b> 成员 | <b>protected</b> 成员 |
| <b>private</b> 成员   | 不可访问                |

| 基类成员访问<br>控制权限 | 派生类 <b>中</b> | 派生类 <b>外</b> |
|----------------|--------------|--------------|
| 公有             | 可以使用         | 可以使用         |
| 保护             | 可以使用         | <b>不可使用</b>  |
| 私有             | <b>不可使用</b>  | <b>不可使用</b>  |



```
#include<iostream.h>
```

```
class Human
```

```
{ public:      int age;
```

```
};
```

```
class Student :public Human
```

```
{public:      void showAge()
```

```
{cout<<"the student age is "<<age<<endl;}
```

```
};
```

```
void main()
```

```
{      Student jessic;
```

```
jessic.age=12;
```

```
jessic.showAge(); }
```

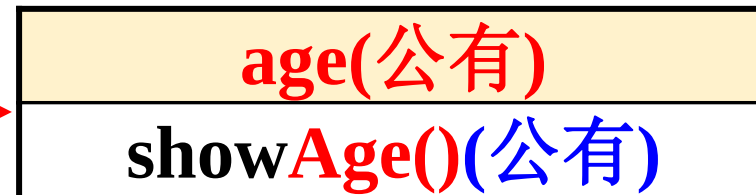
Human

|         |
|---------|
| age(公有) |
|---------|

public



Student



验证:

基类public成员被公有继承后, 在派生类中依然为公有成员

请先思考:

本程序编译是否会出现错误?

(对)

```
#include<iostream.h>
```

```
class Human
```

```
{ private:int age;  
};
```

```
class Student :public Human
```

```
{public: void showAge()
```

```
{cout<<"the student age is "<<age<<endl;}
```

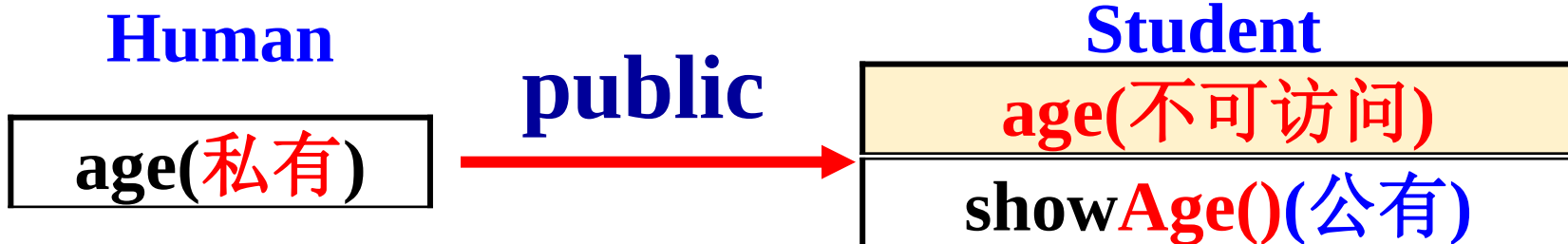
```
};
```

```
void main()
```

```
{ Student jessic;
```

```
jessic.age=12;
```

```
jessic.showAge(); }
```



错误1: 在类中不可访问

验证:

基类private成员被公有继承后, 在派生类中不可访问

请先思考:

如何体现不可访问

错误2: 在类外不可访问

基类的私有成员在派生

在都不可用

增加公有成员函数

```
#include<iostream.h>
```

```
class Human
```

```
{ protected:int age;  
};
```

```
class Student :public Human
```

```
{public:      void showAge()
```

```
{cout<<"the student age is "<<age<<endl;} 无错：在类中可访问
```

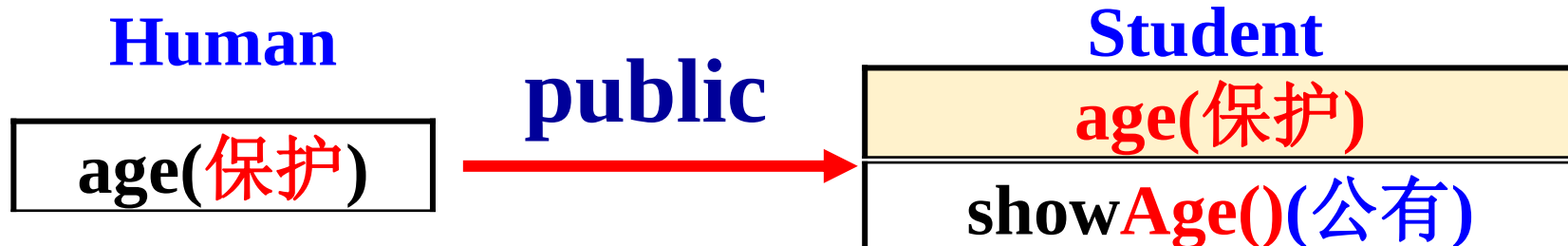
```
};
```

```
void main()
```

```
{      Student jessic;
```

```
jessic.age=12; 错误：在类外不可访问
```

```
jessic.showAge(); }
```



**验证：**

基类protected成员被公有继承后，在派生类中保护

**请先思考：**

如何体现其依然是保护成员

**无法确定是私有还是保护**

```
#include<iostream.h>
```

```
class Human
```

```
{ protected:int age;  
};
```

```
class Student :public Human
```

```
{public: void showAge()  
        {cout<<"the student age is "<<age<<endl;}  
};
```

```
class GStudent :public Student
```

```
{public: void showAge()  
        {cout<<"the Gstudent age is "<<age<<endl;}  
};
```

```
void main()
```

```
{      GStudent jessic;  
        jessic.showAge(); }
```

进一步派生

无错：在类中可访问

说明是保护类型

class A

```
{  
    int x;  
protected: int y;  
public: int z;  
    A(int a,int b,int c){x=a;y=b;z=c;}  
    int Getx(){return x;}  
    int Gety(){return y;}  
    void ShowA()  
    { cout<< "x="<<x<<"\t"<<"y="<<y<<"\t"<<"z="<<z<<"\n"; }
```

class B:public A

```
{  
    int m,n;  
public: B(int a,int b,int c,int d,int e):A(a,b,c)  
    { m=d;n=e;}  
    void Show()  
    { cout<<"m="<<m<<"\t"<<"n="<<n<<"\n";  
      cout<<"x="<<Getx()<<"\t"<<"y="<<Gety()<<"\t"<<"z="<<z<<"\n"; }  
    int Sum(){return ( Getx()+Gety()+z+m+n);} };
```

void main(void)

```
{ B b1(1,2,3,4,5);  
  b1.ShowA();    b1.Show();  
  cout<< "Sum="<<b1.Sum()<<"\n";    cout<<"x="<<b1.Getx()<<"\t";  
  cout << "y=" <<b1.Gety()<<"\t";  cout << "z="<<b1.z<<"\n";    }
```

A

|         |
|---------|
| x(私有)   |
| y(保护)   |
| z(公有)   |
| A()     |
| Getx()  |
| Gety()  |
| ShowA() |

public

B

|              |
|--------------|
| x(不可访问)      |
| y(保护)        |
| z(公有)        |
| Getx() (公有)  |
| Gety() (公有)  |
| ShowA() (公有) |
| m(私有)        |
| n(私有)        |
| B() (公有)     |
| Show() (公有)  |
| Sum() (公有)   |



# 四、派生类成员的访问属性

## 私有继承

| 基类          | 私有派生类     |
|-------------|-----------|
| public成员    | private成员 |
| protected成员 | private成员 |
| private成员   | 不可访问      |

请思考如何用程序验证

| 基类成员访问<br>控制权限 | 派生类中 | 派生类外 |
|----------------|------|------|
| 公有             | 可以使用 | 不可使用 |
| 保护             | 可以使用 | 不可使用 |
| 私有             | 不可使用 | 不可使用 |





# 四、派生类成员的访问属性

## 保护继承

| 基类          | 保护派生类       |
|-------------|-------------|
| public成员    | protected成员 |
| protected成员 | protected成员 |
| private成员   | 不可访问        |

请思考如何用程序验证

| 基类成员访问<br>控制权限 | 派生类中 | 派生类外 |
|----------------|------|------|
| 公有             | 可以使用 | 不可使用 |
| 保护             | 可以使用 | 不可使用 |
| 私有             | 不可使用 | 不可使用 |



## 作业:

请编写测试程序,说明继承方式分别为`private`和`protected`时,基类中的成员 (`public`、`private`和`protected`) 到了派生类后,其访问控制权限的变化, (需要逐条说明)

## 四、派生类成员的访问属性

按访问属性划分为：

- 一、不可访问的成员： 从基类私有成员继承而来。派生类内成员及派生类对象均不能访问。继续派生新类，也无法访问
- 二、私有成员： 新增成员中包括内嵌的其他类的对象。继承或新增而来。类内可访。进一步派生，变成不可访问成员
- 三、保护成员： 继承或新增而来 。派生类内成员能访问。继续派生新类，能成为私有或保护成员。
- 四、公有成员： 继承或新增而来 。派生类内成员及类外均能访问。继续派生新类，能成为新派生类的私有、保护或公有成员



```

class A
{public:      int i;
protected:  void f1();
             int j;
private:     int k;
};

class B:public A
{public:      void f2();
protected:  void f3();
private:     int m;
};

class C:protected B
{public:      void f4();
private:     int n;
};

```

请分析各个成员在不同类中的访问权限

| 类名 | A         | B         | C         |
|----|-----------|-----------|-----------|
| i  | public    | public    | protected |
| f1 | protected | protected | protected |
| j  | protected | protected | protected |
| k  | private   | 不可访问      | 不可访问      |
| f2 |           | public    | protected |
| f3 |           | protected | protected |
| m  |           | private   | 不可访问      |
| f4 |           |           | public    |
| n  |           |           | private   |

## 四、派生类成员的访问属性

不管是按哪一种方式派生，基类的私有成员在派生类中都是不可使用

如果在一个派生类中要访问基类中的私有成员，可以将这个派生类声明为基类的友元。

```
class Base {  
    friend class Derive;  
    ....  
}
```

```
class Derive: public Base  
{  
    ....  
}
```

直接使用Base中的私有成员



## 四、派生类成员的访问属性

```
#include <iostream.h>
```

```
class M
```

```
{ friend class N; //N为M的友元，可以直接使用M中的私有成员
```

```
private:    int i, j;
```

```
        void show(){cout<<"i="<<i<<"\t"<<"j="<<j<<"\t";}
```

```
public:    M(int a=0,int b=0){ i=a; j=b;}
```

```
};
```

```
class N :public M
```

```
{ public:    N(int a=0,int b=0):M(a,b){ }
```

```
        void print(){ show();    cout<<"i+j="<<i+i<<endl; }
```

```
};
```

```
void main(void)
```

```
{  N  n1(10,20);
```

```
    M  m1(100,200);
```

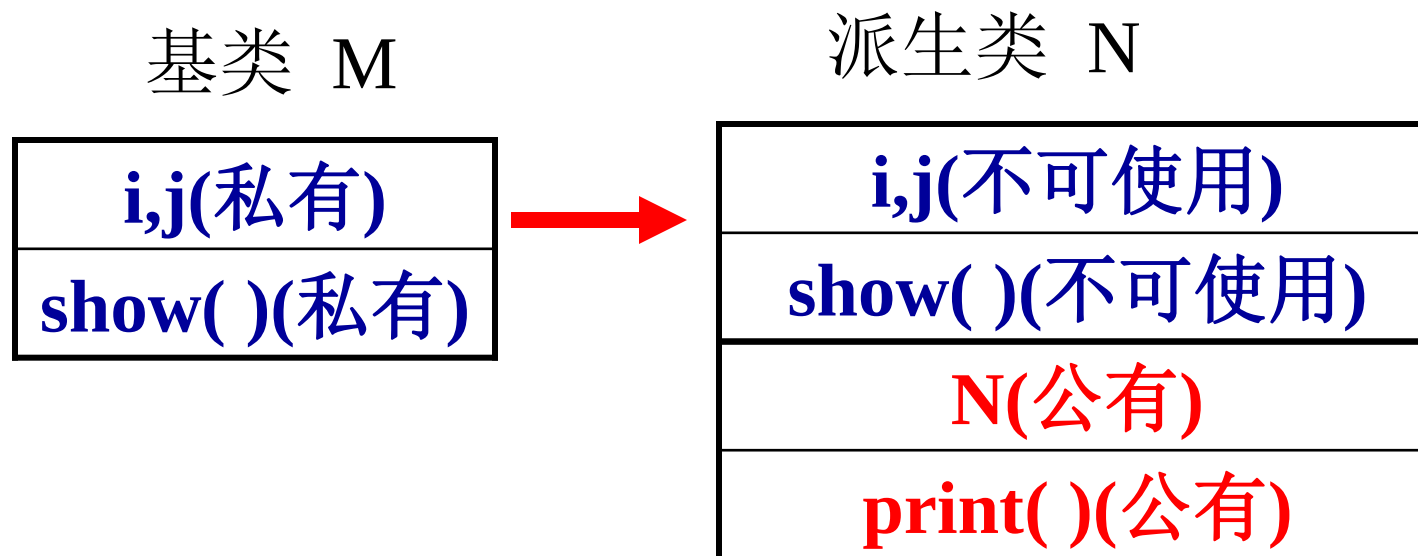
```
    n1.print();
```

```
}
```

直接引用类M的私有成员函数和私有成员



## 四、派生类成员的访问属性



```
print( ){ show(); cout<<i+j<<endl; }
```

当派生类是基类的友元时，上式可以调用



## 四、派生类成员的访问属性

**同名覆盖原则**——当派生类与基类中有相同成员时

- 一、若未强行指明，则通过派生类对象或类内使用的是派生类中的同名成员





## 四、派生类成员函数

```
#include<iostream.h>
class Human
{private:    int age;
public:      void setAge(int a)
            {age=a;}
            int getAge()
            {return age;}
            void showAge()
            {    cout<<"the human age is " <<getAge()<<endl;    }
};
class student :public Human
{public: void showAge()
        {cout<<"the student age is " <<getAge()<<endl;}
};
void main()
{    student jessic;
    jessic.setAge(12);
    jessic.showAge();
}
```



## 四、派生类成员的访问属性

**同名覆盖原则**——当派生类与基类中有相同成员时

- 一、若未强行指明，则通过派生类对象或类内使用的是派生类中的同名成员
- 二、如要通过派生类对象访问基类中被覆盖的同名成员，应使用基类名限定。



## 四、派生类成员

```
#include<iostream.h>
class Human
{
private:    int age;
public:
    void setAge(int a)
    {age=a;}
    int getAge()
    {return age;}
    void showAge()
    {
        cout<<"the human age is "<<getAge()<<endl;

    }
};
class student :public Human
{
public:
    void showAge()
    {cout<<"the student age is "<<getAge()<<endl;}

};
void main()
{
    student jessic;
    jessic.setAge(12);
    jessic.Human::showAge();
}
```



## 四、派生类成员

```
#include<iostream.h>
class Human
{private:  int age;
public:
    void setAge(int a)
    {age=a;}
    int getAge()
    {return age;}
    void showAge()
    {      cout<<"the human age is "<<getAge()<<endl;      }
};
class student :public Human
{
public:
    void showAge()
    {cout<<"the student age is "<<getAge()<<endl;}
};
void main()
{
    Human *hp;
    student jessic;
    hp=&jessic;
    jessic.setAge(12);
    jessic.showAge();
    hp->showAge();
}
```



谢 谢

